

Milestone 2 | Interview Questions

Basic Navigation and File System

Question 1: Finding Your Current Location

Scenario: You just opened the terminal and you're not sure where you are in the file system. Your manager asks you to confirm your current folder location.

Question: Which command will you use and what does it do?

Answer: Use the `pwd` command (Print Working Directory). It shows the complete path of your current location in the file system. For example, if you're in your home folder, it might show `/home/username`.

Question 2: Listing Files in a Directory

Scenario: You need to check what files and folders exist in your project directory before starting work.

Question: What command will you use to see all files and folders?

Answer: Use the `ls` command. It lists all files and folders in your current location. You can also use `ls -l` to see more details like file size, permissions, and date modified.

Question 3: Moving Between Folders

Scenario: Your project has a folder called "backend" and you need to go inside it to check some code files.

Question: How will you navigate into the "backend" folder?

Answer: Use the `cd backend` command (Change Directory). This moves you into the backend folder. To verify, you can use `pwd` to confirm your new location.

Question 4: Going Back to Parent Folder

Scenario: You went into a folder by mistake and need to go back to the previous folder.

Question: Which command helps you move one level up?

Answer: Use `cd ..` command. The two dots (`..`) represent the parent directory (the folder that contains your current folder).

Question 5: Returning to Home Directory

Scenario: You're deep inside multiple nested folders and want to quickly go back to your home folder.

Question: What's the fastest way to go to your home directory?

Answer: Use `cd ~` command. The tilde (`~`) symbol represents your home directory, so this command takes you there directly from anywhere.

File and Folder Creation

Question 6: Creating a New Folder

Scenario: You're starting a new project and need to create a folder called "my-website" to organize your files.

Question: Which command will create this folder?

Answer: Use `mkdir my-website` command (Make Directory). This creates a new folder with the name you specify.

Question 7: Creating Empty Files

Scenario: You need to create a file called "index.html" in your project folder.

Question: How do you create an empty file?

Answer: Use `touch index.html` command. This creates an empty file with the given name. You can then add content to it later using a text editor.

Question 8: Creating Multiple Folders at Once

Scenario: Your project needs three folders: "images", "css", and "scripts". You want to create all three at once.

Question: How can you create multiple folders with a single command?

Answer: Use `mkdir images css scripts` command. You can list multiple folder names separated by spaces, and all will be created together.

File Operations

Question 9: Renaming a File

Scenario: You created a file called "test.txt" but now want to rename it to "notes.txt".

Question: Which command will rename this file?

Answer: Use `mv test.txt notes.txt` command (Move). The `mv` command can both move files and rename them. Here it renames test.txt to notes.txt in the same location.

Question 10: Moving a File to Another Folder

Scenario: You have a file "logo.png" in your current folder, but it should be inside the "images" folder.

Question: How do you move this file?

Answer: Use `mv logo.png images/` command. This moves the logo.png file into the images folder. The slash (/) at the end indicates images is a folder.

Question 11: Deleting a File

Scenario: You have an old file "temp.txt" that you no longer need and want to delete it.

Question: Which command will delete this file?

Answer: Use `rm temp.txt` command (Remove). Be careful with this command because deleted files cannot be easily recovered.

Question 12: Deleting a Folder

Scenario: You have an empty practice folder called "test-folder" that you want to delete.

Question: How do you delete a folder?

Answer: Use `rm -r test-folder` command. The `-r` flag means "recursive" - it deletes the folder and everything inside it. Be very careful with this command!

Viewing and Editing Files

Question 13: Viewing File Contents

Scenario: You want to quickly check what's written inside a file called "config.txt".

Question: Which command shows the file contents?

Answer: Use `cat config.txt` command (Concatenate). It displays the entire content of the file in the terminal window.

Question 14: Viewing Long Files

Scenario: You have a log file with 1000 lines and using `cat` shows too much content at once.

Question: How can you view the file page by page?

Answer: Use `more config.txt` or `less config.txt` command. These let you scroll through the file one page at a time. Press Space to go to the next page and 'q' to quit.

Question 15: Editing a File in Terminal

Scenario: You need to add some text to a file called "readme.txt" directly from the terminal.

Question: Which basic editor can you use?

Answer: Use `nano readme.txt` command. Nano is a simple text editor that works in the terminal. You can type your content, then press Ctrl+O to save and Ctrl+X to exit.

Copying Files and Folders

Question 16: Copying a Single File

Scenario: You want to create a backup of "index.html" and name it "index-backup.html".

Question: How do you copy this file?

Answer: Use `cp index.html index-backup.html` command (Copy). This creates a copy of the file with a new name. The original file remains unchanged.

Question 17: Copying a Folder

Scenario: You need to copy an entire folder called "project" to create a backup called "project-backup".

Question: What command will copy the whole folder with all its contents?

Answer: Use `cp -r project project-backup` command. The `-r` flag stands for "recursive" and is needed to copy folders along with all files and subfolders inside them.

Question 18: Creating Shortcuts (Symbolic Links)

Scenario: You have a configuration file deep inside folders that you need to access frequently. You want to create a shortcut in your home folder.

Question: How do you create a shortcut to this file?

Answer: Use `ln -s /path/to/original/file.txt shortcut-name` command. This creates a symbolic link (like a shortcut in Windows). When you open the shortcut, it points to the original file.

Output Redirection

Question 19: Saving Command Output to a File

Scenario: You want to save the list of all files in your directory into a text file called "file-list.txt".

Question: How do you save the output?

Answer: Use `ls > file-list.txt` command. The `>` symbol redirects the output to a file instead of showing it on the screen. If file-list.txt already exists, it will be overwritten.

Question 20: Adding Output to End of File

Scenario: You have a log file "activity.txt" and want to add today's date to it without deleting the old content.

Question: How do you append (add) to the file?

Answer: Use `date >> activity.txt` command. The `>>` symbol appends the output to the end of the file without deleting existing content. The `>` would replace everything, but `>>` just adds to the end.

Question 21: Creating a File with Content

Scenario: You want to create a new file "message.txt" with the text "Hello World" inside it.

Question: How can you do this in one command?

Answer: Use `echo "Hello World" > message.txt` command. The `echo` command prints text, and `>` saves it to the file. This creates the file and adds the content in one step.

Pipes and Command Chaining

Question 22: Filtering Command Output

Scenario: You have 100 files in a folder but only want to see files with ".txt" in their name.

Question: How do you filter the list?

Answer: Use `ls | grep ".txt"` command. The pipe symbol `|` takes output from `ls` and sends it to `grep`, which searches for lines containing ".txt". This shows only matching files.

Question 23: Counting Files

Scenario: You want to know exactly how many files are in your current folder.

Question: How do you count them using pipes?

Answer: Use `ls | wc -l` command. The `ls` lists all files, the pipe `|` sends that list to `wc -l` (word count with line count option), which counts how many lines (files) there are.

Question 24: Searching and Saving Results

Scenario: You want to find all lines containing the word "error" in a log file and save those lines to a new file.

Question: How do you combine search and save?

Answer: Use `grep "error" app.log > errors.txt` command. This searches for "error" in `app.log` and saves all matching lines to `errors.txt`.

Searching Files and Text

Question 25: Searching Inside a File

Scenario: You have a file "users.txt" with 1000 names and need to check if "Rahul" is in the list.

Question: How do you search for "Rahul"?

Answer: Use `grep "Rahul" users.txt` command. `Grep` searches inside files and shows lines that contain your search word. It will display all lines with "Rahul" in them.

Question 26: Searching Across Multiple Files

Scenario: You want to find which files in your project contain the word "password".

Question: How do you search in all files?

Answer: Use `grep "password" *` command. The asterisk (*) means "all files". `Grep` will search in every file and show which ones contain "password" and the matching lines.

Question 27: Finding Files by Name

Scenario: You remember creating a file with "config" in its name but don't remember the exact name or location.

Question: How do you find this file?

Answer: Use `find . -name "*config*" command`. The dot (.) means start from current folder, `-name` searches by name, and `*config*` finds any filename containing "config" (asterisks mean "anything before or after").

Question 28: Finding and Searching Combined

Scenario: You want to find all ".txt" files that contain the word "important".

Question: How do you combine find and grep?

Answer: Use `find . -name "*.txt" | grep "important"` or `find . -name "*.txt" -exec grep "important" {} \;` command. This finds all .txt files first, then searches inside them for "important".

File Permissions

Question 29: Viewing File Permissions

Scenario: You're getting a "permission denied" error and need to check who can read, write, or execute a file.

Question: How do you see file permissions?

Answer: Use `ls -l filename` command. This shows permissions in format like `rw-r--r--`. First three characters are for owner (read-write), next three for group (read-only), last three for others (read-only).

Question 30: Making a File Readable by Everyone

Scenario: You have a document that currently only you can read, but you want everyone on the team to read it.

Question: How do you change permissions?

Answer: Use `chmod 644 filename` command. The numbers mean: 6 (owner can read and write), 4 (group can only read), 4 (others can only read). Common permission codes are 644 for files and 755 for folders.

Question 31: Making a Script Executable

Scenario: You wrote a script file "backup.sh" but when you try to run it, you get a permission error.

Question: How do you make it executable?

Answer: Use `chmod 755 backup.sh` command. This gives you full permissions (7 = read, write, execute) and others can read and execute (5 = read and execute). Now you can run it with `./backup.sh`.

Question 32: Understanding Permission Numbers

Scenario: Your senior developer asks you to set permissions to "755" on a folder.

Question: What does 755 mean?

Answer:

- First digit (7) = Owner can read(4) + write(2) + execute(1) = 7
- Second digit (5) = Group can read(4) + execute(1) = 5
- Third digit (5) = Others can read(4) + execute(1) = 5

So 755 means owner has full control, but group and others can only read and execute.

Practical Problem Solving

Question 33: Organizing Project Files

Scenario: You downloaded a project and all 50 files are in one folder. You need to organize them - HTML files in one folder, CSS in another, and images in another.

Question: What commands will you use step by step?

Answer:

1. `mkdir html css images` - create three folders
2. `mv *.html html/` - move all HTML files to html folder
3. `mv *.css css/` - move all CSS files to css folder
4. `mv *.png *.jpg images/` - move image files to images folder

Question 34: Creating Project Structure

Scenario: You're starting a new website project and need this structure:

my-website/

- index.html
- css/
 - style.css
- js/
 - app.js
- images/

Question: What commands create this structure?

Answer:

1. `mkdir my-website` - create main folder
2. `cd my-website` - go inside it
3. `touch index.html` - create HTML file
4. `mkdir css js images` - create three folders
5. `touch css/style.css` - create CSS file
6. `touch js/app.js` - create JavaScript file

Question 35: Finding Large Files

Scenario: Your project folder is taking too much space. You need to find which files are largest.

Question: How do you list files by size?

Answer: Use `ls -lh` command to list files with human-readable sizes (KB, MB). Then use `ls -lh | sort -k5 -h` to sort them by size. The `-k5` means sort by 5th column (size) and `-h` means understand human sizes like 5M, 1G.

Question 36: Checking Recent Changes

Scenario: Something broke in your project. You need to see which files were changed in the last hour.

Question: How do you find recently modified files?

Answer: Use `find . -mmin -60` command. The dot (.) means current folder, `-mmin -60` means modified in last 60 minutes. You can also use `ls -lt` to list files sorted by modification time (newest first).

Question 37: Cleaning Up Temporary Files

Scenario: Your project has many temporary files ending with ".tmp" that need to be deleted.

Question: How do you delete all .tmp files at once?

Answer: Use `rm *.tmp` command. The asterisk (*) is a wildcard that matches any filename, so *.tmp matches all files ending with .tmp. Be careful - this will delete all matching files without asking for confirmation!

Question 38: Backing Up Before Making Changes

Scenario: You need to edit an important configuration file but want to keep a backup in case something goes wrong.

Question: What's the safe approach?

Answer:

1. First copy: `cp config.txt config.txt.backup`
2. Then edit: `nano config.txt`
3. If something goes wrong later: `cp config.txt.backup config.txt` to restore

Question 39: Checking Command History

Scenario: You ran a complex command yesterday but can't remember it exactly.

Question: How can you see commands you ran before?

Answer: Use `history` command. This shows all commands you've run. You can also search by using `history | grep "keyword"` to find specific commands. Or press Up arrow key to cycle through previous commands.

Question 40: Clearing Terminal Screen

Scenario: Your terminal is full of old output and you want a clean screen to work with.

Question: How do you clear the terminal?

Answer: Use `clear` command or press `Ctrl+L`. Both clear the screen and give you a fresh workspace. The old output is still there if you scroll up, but you get a clean view.

Advanced Scenarios

Question 41: Monitoring Live Log Files

Scenario: Your application is running and writing logs to "app.log". You want to see new log entries as they appear.

Question: How do you watch a file as it updates?

Answer: Use `tail -f app.log` command. The `tail` shows last few lines of a file, and `-f` flag means "follow" - keep showing new lines as they're added. Press `Ctrl+C` to stop watching.

Question 42: Counting Lines in a File

Scenario: You want to know how many lines of code are in your "app.js" file.

Question: Which command counts the lines?

Answer: Use `wc -l app.js` command. `WC` means "word count", and `-l` flag means count lines only. It will show a number like "250 app.js" meaning 250 lines.

Question 43: Comparing Two Files

Scenario: You have two versions of a file and want to see what changed between them.

Question: How do you compare files?

Answer: Use `diff file1.txt file2.txt` command. It shows lines that are different between the two files. Lines starting with `<` are from first file, lines starting with `>` are from second file.

Question 44: Creating Multiple Nested Folders

Scenario: You need to create this path: "project/src/components/header" but none of these folders exist.

Question: How do you create all folders at once?

Answer: Use `mkdir -p project/src/components/header` command. The `-p` flag means "create parent directories as needed". Without `-p`, you'd get an error because parent folders don't exist.

Question 45: Finding Duplicate Files

Scenario: You suspect you have duplicate image files with different names in your images folder.

Question: How can you find files with same content?

Answer: Use `find images/ -type f -exec md5sum {} \; | sort` command. This creates a unique fingerprint (md5sum) for each file and sorts them. Files with same fingerprint are duplicates. This is advanced but useful for cleaning up.

Question 46: Checking Disk Space

Scenario: Your deployment is failing because the server might be full. You need to check available disk space.

Question: Which command shows disk space?

Answer: Use `df -h` command (Disk Free with human-readable sizes). It shows how much space is used and available on each drive. Look for percentage used - if it's above 90%, you're running out of space.

Question 47: Finding Which Program is Using a File

Scenario: You're trying to delete a file but getting "file is in use" error.

Question: How do you find which program is using it?

Answer: Use `lsof filename` command (List Open Files). It shows which processes have the file open. You can then close that program or kill the process if needed. On some systems, you might need `sudo lsof filename`.

Question 48: Creating Archive of Project

Scenario: You need to send your entire project folder to someone, compressed into a single file.

Question: How do you create a compressed archive?

Answer: Use `tar -czf project.tar.gz project/` command. This creates a compressed (.gz) archive (.tar) file containing your entire project folder. To extract later, use `tar -xzf project.tar.gz`.

Question 49: Finding Empty Files and Folders

Scenario: After cleaning up your project, you want to find and delete any empty files or folders.

Question: How do you find empty items?

Answer:

- For empty files: `find . -type f -empty`
- For empty folders: `find . -type d -empty`
- To delete empty files: `find . -type f -empty -delete`

Be careful with the delete option!

Question 50: Quick File Preview

Scenario: You have 20 log files and want to quickly see the first few lines of each to find the right one.

Question: How do you preview multiple files?

Answer: Use head *.log command. Head shows first 10 lines of a file. When given multiple files, it shows the filename and first few lines of each file, making it easy to scan through multiple files quickly.